

Privacy-Preserving Supervisory Control for Data Opacity Enforcement

Ruochen Tai , Member, IEEE, Liyong Lin , Rong Su , Senior Member, IEEE, and Shuzhi Sam Ge , Fellow, IEEE

Abstract—The privacy-preserving control problem in discrete-event systems at the supervisory control layer is the central focus of this article. The key objective is to cosynthesize an edit function and a supervisor, working together to achieve the following goals: first, ensuring data opacity to prevent external intruders from deducing the system's secret, second, ensuring the system performance adheres to safety and nonblockingness specification, and third, preserving the covert nature of the edit function, creating uncertainty about its presence to external intruders. By transforming this cosynthesis problem into a distributed supervisor synthesis problem in the Ramadge–Wonham supervisory control framework, this article introduces two heuristic synthesis approaches to incrementally cosynthesize an edit function and a supervisor. The effectiveness of the proposed approaches is demonstrated via a running example on location privacy.

Index Terms—Data opacity, edit function, heuristic synthesis, privacy preservation, supervisor.

I. INTRODUCTION

THE increased usage of the Internet and mobile devices has brought about benefits, such as better decision-making, higher productivity, and increased agility. However, challenges cannot be neglected, particularly the security risks posed by cyber intruders targeting sensitive data [1], [2], [3]. These adversaries aim to expose system secrets, which led to the introduction of “opacity” to analyze cryptographic protocols [4]. In the community of discrete-event systems (DES) associated with event-triggered dynamics and discrete-state space [5], [6], which are widely applied in large-scale systems, e.g., automated manufacturing systems, opacity provides a language-based theoretical framework for expressing system security [7]. Intuitively

Received 21 June 2024; revised 14 July 2025; accepted 18 July 2025. Date of publication 10 October 2025; date of current version 5 February 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62503183 and Grant 62320106005, and in part by the National Research Foundation Singapore and DSO National Laboratories under the AI Singapore Programme (AISG) under Award AISG2-GC-2023-007. Paper no. TII-24-3099. (Corresponding author: Ruochen Tai.)

Ruochen Tai is with the School of Artificial Intelligence and Automation, the Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: ruochen_tai@hust.edu.cn).

Liyong Lin and Rong Su are with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore 639798 (e-mail: llin5@e.ntu.edu.sg; rsu@ntu.edu.sg).

Shuzhi Sam Ge is with the Department of Electrical and Computer Engineering, National University of Singapore, Singapore 117576 (e-mail: samge@nus.edu.sg).

Digital Object Identifier 10.1109/TII.2025.3601014

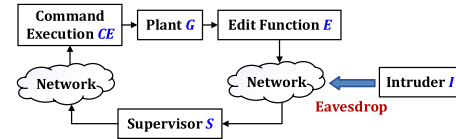


Fig. 1. Privacy-preserving supervisory control architecture.

speaking, opacity evaluates whether external intruders can deduce the secrets by observing the system output. A system is considered opaque if intruders cannot definitively deduce the secrets.

Opacity has been a major focus in the DES community, leading to numerous studies in verification and enforcement. Verification aims to check if the opacity property holds, while enforcement seeks to achieve opacity by integrating privacy-preserving components. This article emphasizes opacity enforcement, which can be achieved by three typical approaches. The first approach involves using supervisory control techniques [7], [8], [9], [10], [11], [12], [13] to regulate the system's behavior and ensure opaque information flow. The second one adopts the edit function [14], [15], [16], [17], [18] to alter the system output to defend against the eavesdropper. The third one is to adopt the (dynamic) mask [19], [20], [21] to (dynamically) turn ON/OFF the associated sensors to influence the observable events to enforce the opacity. Notice that the previous works in the second and third approaches assume the plant behaviors to be free, i.e., they do not realize the closed-loop control. Readers could refer to the survey paper [22] for more details.

Although numerous studies have addressed opacity enforcement in the context of DES, they have overlooked the realistic and emerging privacy-preserving control issue [3] of achieving closed-loop control and preserving privacy-sensitive data over networks [23]. In this work, we concentrate on the supervisory control layer. Our aim is to control the plant to meet safety and nonblocking specification meanwhile the opacity is enforced. We remark that simply using a supervisor to achieve these goals is inadequate because *the intersection of the safety specification and the marked behavior of the plant may lack a nonempty opaque sublanguage*. On the other hand, existing research on edit function (mask) synthesis *does not consider closed-loop control at all*. Thus, simply using an edit function cannot simultaneously achieve opacity enforcement and closed-loop control to realize the privacy-preserving control.

To bridge this research gap, we consider the privacy-preserving control architecture in Fig. 1. As usual, the intruder eavesdrops the sensor information transmitted over the network [7], [8], [9], [10], [13]. The edit function modifies the system output for privacy preservation, and the supervisor issues

control commands based on the modified information for specification fulfillment. In this work, the supervisor and intruder may have different observation sets, i.e., incomparable observations [11], [12]. We remark that the setup where the supervisor observes the edited information instead of the original data is reasonable. First, this setup has been adopted in previous works, e.g., see [24] on homomorphic encryption-based controller design in the classical control, and [25], [26], [27], and [28] on controller encryption in the supervisory control. Second, when the edit function is deployed to modify the sensor information transmitted over the network, ensuring the supervisor's access to original data would require an additional secure channel between the supervisor and plant. This channel must remain hidden from the intruder, who is unaware of its existence. However, such a channel incurs higher costs and efforts in terms of the network construction. By comparison, the setup of Fig. 1 is more economic and feasible. Furthermore, if such a secure channel does exist, then security issues would not occur anymore. In this architecture, our work aims to synthesize an edit function and a supervisor together such that 1) opacity can be enforced and 2) safety and nonblockingness specification can be fulfilled. Thus, our studied problem is not simply an opacity-enforcing supervisory control problem studied in [7], [9], [10], [8], [11], [12], and [13]. It is worth observing that the intruder commonly holds information regarding the system structure. As a result, it could compare its real-time observations with the known structure. If any inconsistency is detected, the existence of the edit function used for opacity enforcement would be exposed to the intruder. To this end, we consider another new goal, called covertness enforcement, requiring that the edit function remains hidden against the intruder.

In the previous works on privacy-preserving control in the realm of DES, the authors in [25] and [26] proposed controller encryption methods by adapting techniques from public-key cryptography. However, they have not tackled the issues of enforcing opacity and covertness. The work in [27] has introduced an event-based cryptographic scheme founded on encryption and decryption functions but does not propose the synthesis approach to generate a suitable pair of encryption and decryption functions. The work in [28] cosynthesizes the dynamic mask, edit function, and supervisor to realize the privacy-preserving control, which is the most relevant to our work. The algorithm in [28] first synthesizes the ensemble of the dynamic mask and edit function, then this ensemble is decomposed into the dynamic mask and edit function, based on which the supervisor is synthesized. The main differences and contributions of this article compared with [28] are summarized as follows.

- 1) We establish a lightweight privacy-preserving control framework where only the edit function and supervisor are adopted to realize the privacy-preserving control. Notice that the editing effects of the dynamic mask and edit function are equal to those of a new edit function. Thus, our lightweight framework maintains the same modeling power with the one in [28]. On the other hand, the complexity of the algorithm in [28] is high due to the decomposition of the dynamic mask and edit function. This is avoided in our framework, and the complexity of synthesizing a privacy-preserving control strategy can be reduced.
- 2) We consider a more challenging realistic scenario where the intruder knows not only the plant but also

the specification. We construct a new intruder model to encode the intruder's prior knowledge and state estimation. Compared with [28] where the intruder only knows the plant, it becomes more difficult to find feasible solutions in our setup since the specification structure would shrink the feasible choices of information editing built upon the plant structure.

- 3) We propose two new algorithms with different synthesis orders for the edit function and the supervisor. In the first algorithm, we first synthesize the supervisor then generate the edit function (the synthesis order is different from that of [28]). To improve the possibility of finding nonempty solutions, two new heuristic strategies are designed, which have not been proposed before. In the second algorithm, we first synthesize the edit function then generate the supervisor, where new plant and requirement structures are constructed in the synthesis procedures to handle the more powerful intruder that knows both the plant and specification. Compared with the algorithm in [28] with the same synthesis order, our second algorithm has a higher possibility to find nonempty solutions because our algorithm does not require the decomposition, while the algorithm in [28] will terminate if the decomposition returns an empty solution. Besides, we can sequentially perform the proposed two algorithms with different synthesis orders, in case one of them return an empty solution, while the algorithm in [28] only adopts a single synthesis order.

The rest of this article is organized as follows. In Section II, some basic notions are provided. In Section III, we construct system models, based on which the privacy-preserving problem is formulated in Section IV. Section V introduces two heuristic incremental synthesis algorithms to synthesize an edit function and a supervisor. Finally, Section VI concludes this article. A running example is given throughout this article.

II. PRELIMINARIES

In this section, following [5] and [6], we introduce some basic notations and terminologies. Σ^* is the Kleene closure for a finite alphabet Σ . The concatenation of two strings $s, t \in \Sigma^*$ is denoted as st . A set of strings is called a language. The natural projection $P_o : \Sigma^* \rightarrow \Sigma_o^*$ is defined as: 1) $P_o(\varepsilon) = \varepsilon$, 2) $(\forall \sigma \in \Sigma) P_o(\sigma) = \sigma$ if $\sigma \in \Sigma_o$; otherwise, $P_o(\sigma) = \varepsilon$, and 3) $(\forall s\sigma \in \Sigma^*) P_o(s\sigma) = P_o(s)P_o(\sigma)$. To explicitly illustrate the codomain Σ_o^* , P_o may also be written as P_{Σ_o} . A finite-state automaton G over Σ is a 5-tuple $(Q, \Sigma, \xi, q^{\text{init}}, Q_m)$, where Q is the state set, $\xi : Q \times \Sigma \rightarrow Q$ is the (partial) transition function, $q^{\text{init}} \in Q$ is the initial state, and Q_m is the set of marker states. If $\xi(q, \sigma)$ is defined, then we write it as $\xi(q, \sigma)!$. $En_G(q) = \{\sigma \in \Sigma \mid \xi(q, \sigma)!\}$. We shall also write $G = (Q, \Sigma, \xi, q^{\text{init}})$ for simplicity if $Q_m = Q$. $L(G)$ and $L_m(G)$ denote the closed behavior and the marked behavior of G [5], respectively. The “unobservable reach” of state $q \in Q$ under the subset $\Sigma' \subseteq \Sigma$ is defined by $UR_{G, \Sigma - \Sigma'}(q) := \{q' \in Q \mid [\exists s \in (\Sigma - \Sigma')^*] q' = \xi(q, s)\}$. We shall abuse the notation and define the subset construction $P_{\Sigma'}(G)$ to be the finite-state automaton $(2^Q, \Sigma, \delta, UR_{G, \Sigma - \Sigma'}(q^{\text{init}}))$, where $\delta : 2^Q \times \Sigma \rightarrow 2^Q$ is defined as follows: 1) for any $\emptyset \neq Q' \subseteq Q$ and $\sigma \in \Sigma'$, $\delta(Q', \sigma) = UR_{G, \Sigma - \Sigma'}(\xi(Q', \sigma))$; and 2) for any $\emptyset \neq Q' \subseteq Q$

and $\sigma \in \Sigma - \Sigma'$, $\delta(Q', \sigma) = Q'$. We denote as $(G)^{|Q-Q'}$ the automaton after removing from G the states in $Q' \subseteq Q$ and the transitions associated with the states in $Q' \subseteq Q$. If $L(G) = \overline{L_m(G)}$, then we say G is nonblocking. If some marker state is reachable in G , then we say G is marker reachable. The parallel composition [6] of two finite-state automata G_1 and G_2 is denoted as $G_1 || G_2$ to capture the joint behavior of G_1 and G_2 .

III. SYSTEM MODELS

In the setup shown in Fig. 1, the control command executed in the plant triggers the firing of the associated events back to the supervisor. The edit function could only edit the events fired by the plant but cannot modify the control commands issued by the supervisor. Consequently, the intruder intercepts the edited event sequences and the supervisor receives the edited sensor observations. In this article, by adopting finite-state automata to model each component in Fig. 1, we target to transform the studied problem to a distributed supervisory control problem. Thus, we shall devote length to the modeling of the components, as presented below.

A. Plant

The plant is a finite-state automaton

$$G = (Q, \Sigma, \xi, q^{\text{init}}, Q_m). \quad (1)$$

We use $Q_{\text{sec}} \subseteq Q$ to denote the secret state set, and $Q_{\text{avoid}} \subseteq Q$ to denote the unsafe state set. As usual, $\Sigma = \Sigma_c \cup \Sigma_{\text{uc}} = \Sigma_o \cup \Sigma_{\text{uo}}$, where Σ_{uc} is the set of uncontrollable events, Σ_c is the set of controllable events, Σ_{uo} is the set of unobservable events, and Σ_o is the set of observable events. In this work, $\Gamma = 2^{\Sigma_c} - \{\emptyset\}$ denotes the control command set. Note that uncontrollable events can be fired at the plant independently of a control command. The specification imposed on G is a finite-state automaton $K = (Q_K, \Sigma_K = \Sigma, \xi_K, q_K^{\text{init}}, Q_{K,m})$ such that 1) $L(K) \subseteq L(G)$, 2) $(\forall t \in L(K)) \xi(q^{\text{init}}, t) \notin Q_{\text{avoid}}$, and 3) $L(K) = \overline{L_m(K)}$. The specification K formulates the legal behavior of the plant G , i.e., the plant behavior under control should be contained in $L(K)$.

Next, we define current-state opacity (CSO), which is our focus (our approach can be potentially extended to other opacity properties).

Definition 1 (See [14], CSO): $G = (Q, \Sigma, \xi, q^{\text{init}}, Q_m)$ is CSO w.r.t. projection P and the set of secret states $Q_{\text{sec}} \subseteq Q$ if for any $t \in \{t \in L(G) | \xi(q^{\text{init}}, t) \in Q_{\text{sec}}\}$, then there exists $t' \in \{t \in L(G) | \xi(q^{\text{init}}, t) \in (Q \setminus Q_{\text{sec}})\}$ such that $P_{\Sigma_o}(t) = P_{\Sigma_o}(t')$.

The CSO says that any string t concluding at a confidential state within Q_{sec} must be accompanied by an alternative sequence t' that concludes at a state within $Q \setminus Q_{\text{sec}}$, and both sequences produce the same observation $P_{\Sigma_o}(t) = P_{\Sigma_o}(t')$. As a result, the intruder is not capable of definitively determining that the system has reached a confidential state currently when observing $P(t)$.

Example 1: We adapt the location privacy example of [29] and consider the scenario where an autonomous vehicle is executing tasks in the wild environment. The autonomous vehicle is controlled by a remote controller to ensure the correct execution of the predefined path planning strategy. The remote sensing map for wild environment is shown in Fig. 2, which is discretized by selecting 13 locations (circles) that the vehicle needs to pass as states, and the connection routes

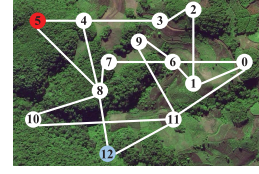


Fig. 2. Remote sensing map of the wild environment where an autonomous vehicle executes tasks.

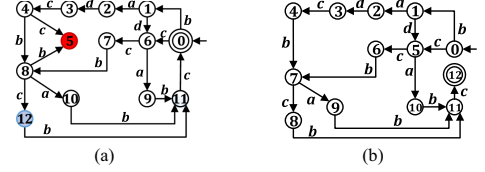


Fig. 3. (a) Plant G . (b) Specification K .

(white lines) between these locations as transitions. The vehicle starts from location 0, i.e., $q^{\text{init}} = 0$. Location 12 is the secret state, i.e., $Q_{\text{sec}} = \{12\}$. Location 5 is the unsafe state that the vehicle should not reach, i.e., $Q_{\text{avoid}} = \{5\}$. The event set is modeled as the vehicle's action event set, denoted as $\Sigma = \{a, b, c, d\}$. The event a denotes for turning right, b for turning left, c for going straight, and d for rushing down the steep slope uncontrollably. Thus, $\Sigma_{\text{uc}} = \{d\}$ and $\Sigma_c = \{a, b, c\}$. We then have $\Gamma = \{\gamma_1, \gamma_2, \gamma_3, \gamma_4, \gamma_5, \gamma_6, \gamma_7\}$, where $\gamma_1 = \{a\}$, $\gamma_2 = \{b\}$, $\gamma_3 = \{c\}$, $\gamma_4 = \{a, b\}$, $\gamma_5 = \{a, c\}$, $\gamma_6 = \{b, c\}$, and $\gamma_7 = \{a, b, c\}$. In this location privacy example, all the action events are observable, i.e., $\Sigma_o = \Sigma$. Based on the modeled states and events, the plant is formulated in Fig. 3(a), describing the transition relation between locations and vehicle actions. The specification K is illustrated in Fig. 3(b). Notice that K is only related to the legal behavior of G and does not involve the opacity requirement. Besides, the state number of G and K are independent. Thus, the state 12 of G does not correspond to the state 12 of K . According to definition 1, CSO could be violated, e.g., the execution of the string $badebc$ by the vehicle would expose that the secret location 12 is reached, which urges the need for privacy-preserving control.

B. Edit Function

The edit function is able to modify the events fired by the plant [16]. Its observable event set is denoted as $\Sigma_{o,E} \subseteq \Sigma_o$ and editable event set is denoted as $\Sigma_{s,E} \subseteq \Sigma_{o,E}$. The edit function can perform operations, including deletion, insertion, and replacement only w.r.t. $\Sigma_{s,E}$ [16]. In practice, it is very likely that $\Sigma_{s,E} = \Sigma_{o,E} = \Sigma_o$, but here we adopt the setup where $\Sigma_{s,E} \subseteq \Sigma_{o,E} \subseteq \Sigma_o$ for the sake of generality. We assume that any edited sequence is bounded by U [16] after an observation of the edit function.

In this work, to capture the editing effects on the sensor observations to be received by the supervisor, we make a relabeled copy $\Sigma_{s,E}^\# = \{\sigma^\# | \sigma \in \Sigma_{s,E}\}$ of $\Sigma_{s,E}$ such that events in $\Sigma_{s,E}$ are executed by the plant, while events in $\Sigma_{s,E}^\#$ are those edited copies sent by the edit function and received by the supervisor. In addition, for the supervisor model presented later in Section II-I-C, the supervisor's observations come from two sources, where one is the original version of observable noneditable events in $\Sigma_o - \Sigma_{s,E}$ and another is the relabeled observable editable

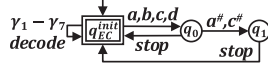


Fig. 4. Edit constraint EC ($\gamma_1 - \gamma_7$ means $\gamma_1, \gamma_2, \gamma_3, \gamma_4, \gamma_5, \gamma_6, \gamma_7$).

events in $\Sigma_{s,E}^\#$. In other words, the supervisor could only react to events in $(\Sigma_o - \Sigma_{s,E}) \cup \Sigma_{s,E}^\#$ instead of Σ_o .

Next, we introduce two models. One is the EC to formulate the edit function capability and another is the edit function to be synthesized.

EC: The EC is modeled as a finite-state automaton $EC = (Q_{EC}, \Sigma_{EC}, \xi_{EC}, q_{EC}^{init}, Q_{EC,m})$, where $Q_{EC} = \{q_n | n \in [0 : U]\} \cup \{q_{EC}^{init}\}$, $\Sigma_{EC} = \Sigma \cup \Sigma_{s,E}^\# \cup \Gamma \cup \{\text{stop, decode}\}$, $Q_{EC,m} = \{q_{EC}^{init}\}$, and $\xi_{EC} : Q_{EC} \times \Sigma_{EC} \rightarrow Q_{EC}$ is the (partial) transition function defined as follows.

- 1) $(\forall \sigma \in (\Sigma - \Sigma_{o,E}) \cup \Gamma \cup \{\text{decode}\}) \xi_{EC}(q_{EC}^{init}, \sigma) = q_{EC}^{init}$.
- 2) $(\forall \sigma \in \Sigma_{o,E}) \xi_{EC}(q_{EC}^{init}, \sigma) = q_0$.
- 3) $(\forall n \in [0 : U - 1]) (\forall \sigma \in \Sigma_{s,E}) \xi_{EC}(q_n, \sigma^\#) = q_{n+1}$.
- 4) $(\forall n \in [0 : U]) \xi_{EC}(q_n, \text{stop}) = q_{EC}^{init}$.

In the state set Q_{EC} , there are states denoted as q_n where $n \in [0 : U]$. Each state q_n signifies that n events have been sent by the edit function upon observation from $\Sigma_{o,E}$. The event set Σ_{EC} is a combination of several subsets. Σ is the plant event set. $\Sigma_{s,E}^\#$ contains events denoted by $\sigma^\#$ ($\sigma \in \Sigma_{s,E}$), which denotes that an editable event σ is sent by the edit function. Γ contains control commands. $\{\text{stop, decode}\}$ contains the event denoting the end of the current round of edit operation, symbolized by stop, which can be controlled and observed by the edit function, and the event denoting privacy leakage, denoted by decode. We will explain the event decode later. It is emphasized that the supervisor is only allowed to observe and react to events in $\Sigma_{s,E}^\#$ rather than original events in $\Sigma_{s,E}$. By this way, the editing effects can be captured.

The following cases are defined in the transition function ξ_{EC} .

Case 1: At state q_{EC}^{init} , if any event σ occurs such that $\sigma \in (\Sigma - \Sigma_{o,E}) \cup \Gamma \cup \{\text{decode}\}$, any edit operation will not be performed, formalized by a self-loop, since σ cannot be observed by the edit function. **Case 2:** At state q_{EC}^{init} , any observation of $\sigma \in \Sigma_{o,E}$ results in the transition to state q_0 . In this new state, replacing σ with any editable event from the set $\Sigma_{s,E}$ or deleting σ is allowed. **Case 3:** The edit function can insert any editable event $\sigma \in \Sigma_{s,E}$, denoted by $\sigma^\#$, at any state q_n , where $n \in [0 : U - 1]$. It is important to note that the insertion sequence bound is U . Hence, at state q_U , the edit function is not permitted to insert any events in $\Sigma_{s,E}$. **Case 4:** The edit function could decide when to end the editing, denoted by stop, at any state q_n ($n \in [0 : U]$), and returns to the state q_{EC}^{init} .

Example 2: Let us continue Example 1. $\Sigma_{o,E} = \{a, b, c, d\}$. $\Sigma_{s,E} = \{a, c\}$. For simplicity, we set $U = 1$. EC is shown in Fig. 4. The event editing capability is explained as follows by taking the observation of a and c as an instance: 1) at state q_0 , the observed event a (turning right) could be kept intact, denoted by $a^\#$, or altered into c (going straight), denoted by the transition labeled as $c^\#$, 2) at state q_0 , the observed event c (going straight) could be kept intact, denoted by $c^\#$, or altered into a (turning right), denoted by $a^\#$, and 3) at state q_0 , the observed event a (turning right) or c (going straight) could be deleted, denoted by stop.

Remark 1: The value of U depends on the user design. Here, we analyze the effects of different U to guide its choice. A larger U indicates that the edit function has more choices to modify the sensor information, i.e., the solution space becomes larger. Consequently, the success rate of finding nonempty solutions in general turns higher. However, when U increases, more computation time is needed to search for feasible solutions (the complexity analysis is presented later in Section V). Thus, how to find the optimal U such that we can achieve the tradeoff between the computation efficiency and the success rate of finding nonempty solutions is a topic worthy of investigation, which is left as future works.

Edit Function: The edit function is modeled as a finite-state automaton $E = (Q_E, \Sigma_E, \xi_E, q_E^{init}, Q_{E,m})$. $\Sigma_E = \Sigma_{EC} = \Sigma \cup \Sigma_{s,E}^\# \cup \Gamma \cup \{\text{stop, decode}\}$. Two constraints should be satisfied by E : 1) (E -observability) $(\forall q \in Q_E) (\forall \sigma \in \Sigma_E - \Sigma_{E,obs} = \Sigma_E - (\Sigma_{o,E} \cup \Sigma_{s,E}^\# \cup \{\text{stop}\})) \xi_E(q, \sigma) \Rightarrow \xi_E(q, \sigma) = q$; and 2) (E -controllability) $(\forall q \in Q_E) (\forall \sigma \in \Sigma_E - \Sigma_{E,con} = \Sigma_E - (\Sigma_{s,E}^\# \cup \{\text{stop}\})) \xi_E(q, \sigma) \neq \text{stop}$. The first constraint says that only an observation from $\Sigma_{E,obs} = \Sigma_{o,E} \cup \Sigma_{s,E}^\# \cup \{\text{stop}\}$ could trigger a state change. The second constraint says that only events in $\Sigma_{E,con} = \Sigma_{s,E}^\# \cup \{\text{stop}\}$ can be disabled by the edit function. We shall refer to $(\Sigma_{E,con}, \Sigma_{E,obs})$ as the edit function-control constraint in the following text.

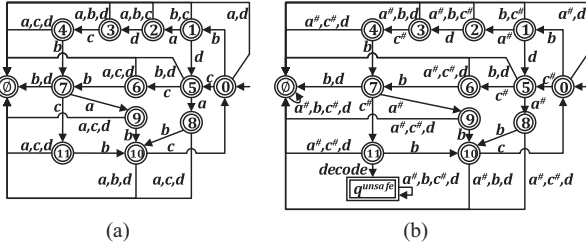
C. Supervisor

We introduce two models in this section. One is the SC to formulate the supervisor capability and another is the supervisor to be synthesized.

Supervisor constraint: Due to the event relabeling w.r.t. the set $\Sigma_{s,E}$, the set of observed events by the supervisor becomes $(\Sigma_o - \Sigma_{s,E}) \cup \Sigma_{s,E}^\#$. Thus, the supervisor can only send a new control command after receiving the observation either from the original set of observable noneditable events $\Sigma_o - \Sigma_{s,E}$ or from the edited set $\Sigma_{s,E}^\#$. In this work, control commands are observable to the supervisor as the supervisor should be aware of its own actions. The SC is constructed as a finite-state automaton $SC = (Q_{SC}, \Sigma_{SC}, \xi_{SC}, q_{SC}^{init})$, where $Q_{SC} = \{q_{SC}^{init}, q_{SC}^{obs}\}$, $\Sigma_{SC} = \Sigma \cup \Sigma_{s,E}^\# \cup \Gamma \cup \{\text{stop, decode}\}$, and $\xi_{SC} : Q_{SC} \times \Sigma_{SC} \rightarrow Q_{SC}$ is the (partial) transition function defined as follows.

- 1) $(\forall \sigma \in \Sigma_{SC} - \Gamma) \xi_{SC}(q_{SC}^{init}, \sigma) = q_{SC}^{init}$.
- 2) $(\forall \gamma \in \Gamma) \xi_{SC}(q_{SC}^{init}, \gamma) = q_{SC}^{obs}$.
- 3) $(\forall \sigma \in (\Sigma_o - \Sigma_{s,E}) \cup \Sigma_{s,E}^\#) \xi_{SC}(q_{SC}^{obs}, \sigma) = q_{SC}^{init}$.
- 4) $(\forall \sigma \in \Sigma_{uo} \cup \Sigma_{s,E} \cup \{\text{stop, decode}\}) \xi_{SC}(q_{SC}^{obs}, \sigma) = q_{SC}^{obs}$.

In the state set Q_{SC} , q_{SC}^{init} denotes either the system initiation or the case where an observation is received. q_{SC}^{obs} denotes that a control command has just been issued. Regarding the (partial) transition function ξ_{SC} , the following cases are defined. **Cases 1 and 2:** At q_{SC}^{init} , if a control command is issued, then SC transits to q_{SC}^{obs} , while the state does not change if any other event $\sigma \in \Sigma_{SC} - \Gamma$ occurs. It means that after an observation, the state transition can happen only when a control action is implemented. **Case 3:** At q_{SC}^{obs} , SC turns back to the initial state q_{SC}^{init} if an event $\sigma \in (\Sigma_o - \Sigma_{s,E}) \cup \Sigma_{s,E}^\#$ occurs. It means that after a control command is issued, the state transition can happen only when some observation from the set $(\Sigma_o - \Sigma_{s,E}) \cup \Sigma_{s,E}^\#$

Fig. 7. (a) H . (b) Intruder $\mathcal{I}$.

Step 2: Next, we construct the intruder model by modifying H to explicitly encode the privacy leakage scenarios and event editing effects. We denote the intruder model as $\mathcal{I} = (Q_{\mathcal{I}}, \Sigma_{\mathcal{I}}, \xi_{\mathcal{I}}, q_{\mathcal{I}}^{\text{init}})$, where $Q_{\mathcal{I}} = Q_H \cup \{q^{\text{unsafe}}\}$, $\Sigma_{\mathcal{I}} = (\Sigma_H - \Sigma_{s,E}) \cup \Sigma_{s,E}^{\#} \cup \{\text{decode}\} = (\Sigma - \Sigma_{s,E}) \cup \Sigma_{s,E}^{\#} \cup \{\text{decode}\}$, $q_{\mathcal{I}}^{\text{init}} = q_H^{\text{init}}$, and $\xi_{\mathcal{I}} : Q_{\mathcal{I}} \times \Sigma_{\mathcal{I}} \rightarrow Q_{\mathcal{I}}$ is the (partial) transition function defined as follows.

- 1) $(\forall q \in Q_{\mathcal{I}}) q \in 2^{Q_{\text{sec}} \times Q_K} - \{\emptyset\} \Rightarrow \xi_{\mathcal{I}}(q, \text{decode}) = q^{\text{unsafe}}$.
- 2) $(\forall q, q' \in Q_{\mathcal{I}})(\forall \sigma \in \Sigma_{s,E}) \xi_H(q, \sigma) = q' \Rightarrow \xi_{\mathcal{I}}(q, \sigma^{\#}) = q'$.
- 3) $(\forall q, q' \in Q_{\mathcal{I}})(\forall \sigma \in \Sigma - \Sigma_{s,E}) \xi_H(q, \sigma) = q' \Rightarrow \xi_{\mathcal{I}}(q, \sigma) = q'$.
- 4) $(\forall q \in \{\emptyset, q^{\text{unsafe}}\})(\forall \sigma \in (\Sigma - \Sigma_{s,E}) \cup \Sigma_{s,E}^{\#}) \xi_{\mathcal{I}}(q, \sigma) = q$.

The intruder model $\mathcal{I}$ introduces a new event decode and a new state q^{unsafe} , which symbolizes the dilemma that reaching a secret state has been inferred by the intruder. To depict this scenario, a new outgoing transition $\xi_{\mathcal{I}}(q, \text{decode}) = q^{\text{unsafe}}$ is attached to state $q \in 2^{Q_{\text{sec}} \times Q_K} - \emptyset$. During Steps 2 and 3, any transitions labeled with events from $\Sigma_{s,E}$ undergo substitution with their corresponding relabeled copies in $\Sigma_{s,E}^{\#}$ to encode the effects of event editing. Notice that we do not change other transitions. In Step 4, at either state $\emptyset$ or q^{unsafe} , any event in the set $(\Sigma - \Sigma_{s,E}) \cup \Sigma_{s,E}^{\#}$ is self-looped. We made such a self-loop because, regardless of any event occurrence, the fact of the privacy leakage or the exposure of the edit function remains unaffected.

Remark 3: Our modeling technique could be extended to the case where the intruder eavesdrops sensor events and control commands, with some minor modifications. Specifically speaking, 1) in step 1, $P_{\Sigma_{o,\mathcal{I}}}(G||K)$ is replaced with $P_{\Sigma_{o,\mathcal{I}} \cup \mathcal{I}}(G||K||\text{CE})$, encoding the sequences of commands and events that should have been observed without editing, and 2) in case 1 of $\xi_{\mathcal{I}}$ in step 2, $q \in 2^{Q_{\text{sec}} \times Q_K}$ is replaced with $q \in 2^{Q_{\text{sec}} \times Q_K \times Q_{\text{CE}}}$. The proposed two synthesis algorithms (see Section V) are still applicable to this new intruder.

Example 5: We continue the running example. Following the aforementioned Steps 1 and 2, the structure H ($\Sigma_{o,\mathcal{I}} = \{a, b, c, d\}$) and intruder $\mathcal{I}$ are constructed and illustrated in Fig. 7. Due to the event relabeling effect, the intruder $\mathcal{I}$ would only react to $a^{\#}$, $b^{\#}$, and d since $a, c \in \Sigma_{s,E}$. At the initial state 0, only b (turning left) and c (going straight) could happen according to the structure of G and K , respectively. Thus, if the intruder observes $a^{\#}$ (turning right) and d (rushing down the slope) at the initial state 0, it asserts that the information inconsistency is detected and the edit function is discovered. Such an information inconsistency is denoted by the transitions at the state 0, labeled as $a^{\#}$ and d , which lead to the state $\emptyset$. At

the state 11, the intruder is assured that the vehicle has reached the secret location 12. Hence, there exists a transition labeled as decode, leading to the privacy-leakage state q^{unsafe} .

IV. PROBLEM FORMULATION

According to the definition of the parallel composition “||”, the closed-loop system, as shown in Fig. 1, can be treated as the joint behavior of the above system models, which is denoted by $\mathcal{B} = G||\text{CE}||\text{EC}||\text{SC}||\mathcal{I}||E||S = (Q_{\mathcal{B}}, \Sigma_{\mathcal{B}}, \xi_{\mathcal{B}}, q_{\mathcal{B}}^{\text{init}}, Q_{\mathcal{B},m})$. We shall refer to the edit function E combined with the supervisor S as an edit function–supervisor pair (E, S) in the following text.

Definition 2 (Opacity): Given G , CE, EC, SC, and $\mathcal{I}$, the edit function–supervisor pair (E, S) is *opaque* if $\mathcal{B} = G||\text{CE}||\text{EC}||\text{SC}||\mathcal{I}||E||S$ does not reach any state in $\{(q, q_{\text{ce}}, q_{\text{ec}}, q_{\text{sc}}, q_i, q_e, q_s) \in Q_{\mathcal{B}} | q_i = q^{\text{unsafe}}\}$.

Definition 3 (Covertness): Given G , CE, EC, SC, and $\mathcal{I}$, the edit function–supervisor pair (E, S) is *covert* if $\mathcal{B} = G||\text{CE}||\text{EC}||\text{SC}||\mathcal{I}||E||S$ does not reach any state in $\{(q, q_{\text{ce}}, q_{\text{ec}}, q_{\text{sc}}, q_i, q_e, q_s) \in Q_{\mathcal{B}} | q_i = \emptyset\}$.

Problem 1: Given G , K , CE, EC, SC, $\mathcal{I}$, $(\Sigma_{E,\text{con}}, \Sigma_{E,\text{obs}})$, and $(\Sigma_{S,\text{con}}, \Sigma_{S,\text{obs}})$, synthesize the edit function and supervisor such that: 1) $\mathcal{B}$ is nonblocking, 2) $P_{\Sigma}(L(\mathcal{B})) \subseteq L(K)$, and 3) the edit function–supervisor pair is opaque and covert.

V. HEURISTIC INCREMENTAL SYNTHESIS ALGORITHMS

Since $\mathcal{B} = G||\text{CE}||\text{EC}||\text{SC}||\mathcal{I}||E||S$ denotes the closed-loop system, we treat E and S as the distributed supervisors to be synthesized over the distributed control architecture $\mathcal{A} = ((\Sigma_{E,\text{con}}, \Sigma_{E,\text{obs}}), (\Sigma_{S,\text{con}}, \Sigma_{S,\text{obs}}))$ of the new plant $\mathcal{P} = G||\text{CE}||\text{EC}||\text{SC}||\mathcal{I}$. Note that the distributed supervisor synthesis problem is in general *undecidable* [30]. Thus, our work exploits the unique structure of the modeled distributed supervisor synthesis problem and presents two heuristics to compute the edit function and supervisor for attaining safety, opacity, covertness, and nonblockingness objectives. The first heuristic involves synthesizing the supervisor initially and then proceeding to synthesize the edit function, while the second heuristic follows the opposite order by first generating the edit function and then synthesizing the supervisor.

A. Incremental Synthesis: First S Then E

The synthesis procedure begins with the generation of the supervisor to guarantee the plant safety in compliance with the specification as well as the marker reachability of the closed-loop system. Once we successfully synthesize a supervisor, the next part involves the synthesis of the edit function, where the objective is to achieve opacity, covertness, and nonblockingness properties for the overall system. The details of the proposed algorithm are outlined as follows.

Steps 1–8 focus on the synthesis of the supervisor while Steps 9–11 pertain to the synthesis of the edit function. Specifically, in Steps 1–3 related to synthesizing the supervisor, two main components are involved: $\mathcal{P}$ and $\mathcal{R}_S^0$. Here, $\mathcal{P}$ represents the plant, and $\mathcal{R}_S^0$ represents the initial requirement, derived by removing the set of states denoted by Q_1 from $\mathcal{R}$. Here, $\mathcal{R}$ is generated in step 2 by combining the imposed specification K with $\mathcal{P}$, denoted by $\mathcal{R} = G||K||\text{CE}||\text{EC}||\text{SC}||\mathcal{I}$. We remark that it is necessary to consider the specification K in the synthesis of supervisor because the supervisor is the only component that could satisfy K by sending appropriate control commands. In the removed state set Q_1 , these states are eliminated because they

Procedure 1.

Input: $G, CE, EC, SC, I, K, (\Sigma_{E,con}, \Sigma_{E,obs}), (\Sigma_{S,con}, \Sigma_{S,obs})$.

Output: E, S .

1. Compute

$$\mathcal{P} = G||CE||EC||SC||I = (Q_{\mathcal{P}}, \Sigma_{\mathcal{P}}, \xi_{\mathcal{P}}, q_{\mathcal{P}}^{\text{init}}, Q_{\mathcal{P},m}).$$

2. Compute

$$\mathcal{R} = G||K||CE||EC||SC||I = (Q_{\mathcal{R}}, \Sigma_{\mathcal{R}}, \xi_{\mathcal{R}}, q_{\mathcal{R}}^{\text{init}}, Q_{\mathcal{R},m})$$

and generate $\mathcal{R}_S^0 := (\mathcal{R})^{Q_{\mathcal{R}} - Q_1} = (Q_{\mathcal{R}_S^0}, \Sigma_{\mathcal{R}_S^0}, \xi_{\mathcal{R}_S^0}, q_{\mathcal{R}_S^0}^{\text{init}}, Q_{\mathcal{R}_S^0,m})$.

$$\bullet Q_1 := \{(q, q_k, q_{ce}, q_{ec}, q_{sc}, q_i) \in Q_{\mathcal{R}} \mid q \notin Q_m \wedge q_{ce} = q_{ce}^{\text{init}} \wedge (\gamma \cup \Sigma_{uc}) \cap \text{En}_G(q) = \emptyset\}.$$

3. Synthesize a supervisor $S_0 = (Q_{S_0}, \Sigma_{S_0}, \xi_{S_0}, q_{S_0}^{\text{init}}, Q_{S_0,m})$ over the supervisor-control constraint $(\Sigma_{S,con}, \Sigma_{S,obs})$ by treating $\mathcal{P}$ as the plant and $\mathcal{R}_S^0$ as the requirement such that $\mathcal{P}||S_0$ is marker-reachable and $L(\mathcal{P}||S_0) \subseteq L(\mathcal{R}_S^0)$. If S_0 exists, go to Step 4; otherwise, end the procedure.

4. Let $j \leftarrow 0$.

5. Compute $\mathcal{R}||S_j = (Q_{\mathcal{R}||S_j}, \Sigma_{\mathcal{R}||S_j}, \xi_{\mathcal{R}||S_j}, q_{\mathcal{R}||S_j}^{\text{init}}, Q_{\mathcal{R}||S_j,m})$.

6. Generate $Q_{del} := \{q \in Q_{\mathcal{R}||S_j} \mid C_1 \vee C_2\}$, where

$$(\exists (q, q_s) \in Q_{\mathcal{R}||S_j}) (\forall t \in \Sigma_{\mathcal{R}||S_j}^*) \xi_{\mathcal{R}||S_j}((q, q_s), t)$$

$$C_1 := \notin Q_m \times Q_K \times Q_{CE} \times Q_{EC} \times Q_{SC} \times Q_I \times Q_{S_j}$$

$$C_2 := (\forall q_s \in Q_{S_j}) (q, q_s) \notin Q_{\mathcal{R}||S_j}$$

If $Q_{del} \neq \emptyset$, go to Step 7; otherwise, i.e., $Q_{del} = \emptyset$, go to Step 9 and treat S_j as the desired supervisor S , denoted as $S := S_j = (Q_S, \Sigma_S, \xi_S, q_S^{\text{init}}, Q_{S,m})$.

7. Generate $\mathcal{R}_S^{j+1} := (\mathcal{R}_S^j)^{Q_{\mathcal{R}_S^j} - Q_{del}} = (Q_{\mathcal{R}_S^{j+1}}, \Sigma_{\mathcal{R}_S^{j+1}}, \xi_{\mathcal{R}_S^{j+1}}, q_{\mathcal{R}_S^{j+1}}^{\text{init}}, Q_{\mathcal{R}_S^{j+1},m})$.

8. Synthesize a supervisor

$S_{j+1} = (Q_{S_{j+1}}, \Sigma_{S_{j+1}}, \xi_{S_{j+1}}, q_{S_{j+1}}^{\text{init}}, Q_{S_{j+1},m})$ over the supervisor-control constraint $(\Sigma_{S,con}, \Sigma_{S,obs})$ by treating $\mathcal{P}$ as the plant and $\mathcal{R}_S^{j+1}$ as the requirement such that $\mathcal{P}||S_{j+1}$ is marker-reachable and $L(\mathcal{P}||S_{j+1}) \subseteq L(\mathcal{R}_S^{j+1})$. If S_{j+1} exists, let $j \leftarrow j + 1$ and go to Step 5; otherwise, end the procedure.

9. Compute $\mathcal{P}_E = G||CE||EC||SC||I||S = (Q_{\mathcal{P}_E}, \Sigma_{\mathcal{P}_E}, \xi_{\mathcal{P}_E}, q_{\mathcal{P}_E}^{\text{init}}, Q_{\mathcal{P}_E,m})$.

10. Generate $\mathcal{R}_E^0 := (\mathcal{P}_E)^{Q_{\mathcal{P}_E} - Q_2} = (Q_{\mathcal{R}_E^0}, \Sigma_{\mathcal{R}_E^0}, \xi_{\mathcal{R}_E^0}, q_{\mathcal{R}_E^0}^{\text{init}}, Q_{\mathcal{R}_E^0,m})$.

$$\bullet Q_2 := \{(q, q_{ce}, q_{ec}, q_{sc}, q_i, q_s) \in Q_{\mathcal{P}_E} \mid q_i = q^{\text{unsafe}} \vee q_i = \emptyset\}.$$

11. Synthesize a supervisor $E = (Q_E, \Sigma_E, \xi_E, q_E^{\text{init}}, Q_{E,m})$ over the edit function-control constraint $(\Sigma_{E,con}, \Sigma_{E,obs})$ by treating $\mathcal{P}_E$ as the plant and $\mathcal{R}_E^0$ as the requirement such that $\mathcal{P}_E||E$ is nonblocking and $L(\mathcal{P}_E||E) \subseteq L(\mathcal{R}_E^0)$.

represent deadlocked $((\gamma \cup \Sigma_{uc}) \cap \text{En}_G(q) = \emptyset)$, nonmarked $(q \notin Q_m)$ situations where the plant cannot use control commands sent by the supervisor. In Step 3, the supervisor S_0 is computed based on the models $\mathcal{P}$ and $\mathcal{R}_S^0$. The primary objectives of this computation are twofold: 1) ensuring $L(\mathcal{P}||S_0) \subseteq L(\mathcal{R}_S^0)$, and 2) achieving marker reachability. It is worthy to point out that achieving nonblockingness in the closed-loop system can be difficult at this stage due to the uncontrollable nature of the edit operations to the supervisor.

Despite the potential for S_0 to ensure the reachability of certain marker states in $\mathcal{P}||S_0$, it is essential to recognize that, under

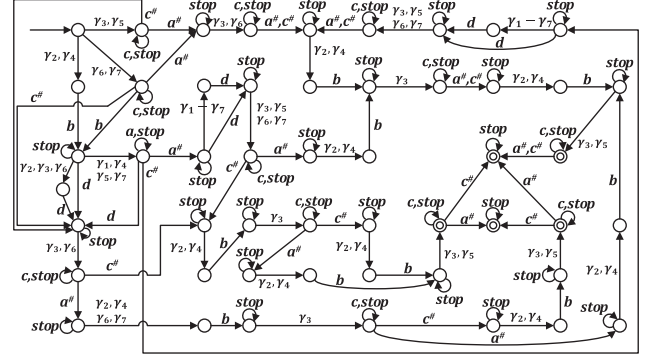


Fig. 8. Synthesized S (first S then E).

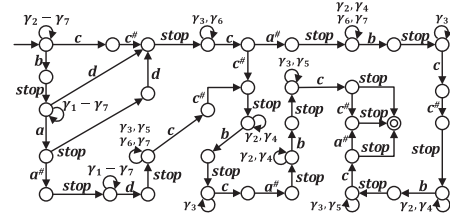


Fig. 9. Synthesized E (first S then E).

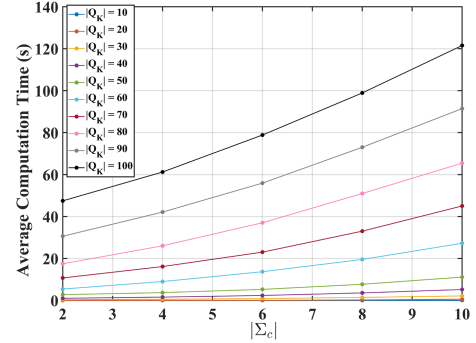


Fig. 10. Average computation time of 20 repeated simulations under different $|Q_K|$ and $|\Sigma_c|$ for Procedure 1.

the control of S_0 , blockingness in the plant G can still occur. This may pose challenges in achieving nonblockingness for the closed-loop system behavior when carrying out the following edit function synthesis based on S_0 as the edit function cannot control the control commands sent by the supervisor. To ease this potential dilemma, we propose a heuristic approach to increase the chances of finding a nonempty edit function later, which involves the iterative computation of a supervisor (Steps 5–8). This process continues until the blockingness no longer occurs in the plant. Notice that in each iterative synthesis, we need to identify the state set Q_{del} (see Step 6) that needs to be removed, any state q of which satisfies C_1 (blockingness in the plant) or C_2 (nonreachability of q).

During the edit function synthesis (Steps 9–11), we construct the surrogate plant $\mathcal{P}_E$ and the surrogate requirement $\mathcal{R}_E^0$. The composition $G||CE||EC||SC||I$ and the supervisor S synthesized in Steps 1–8 generate $\mathcal{P}_E$, from which we eliminate states corresponding to the situations where either privacy leakage ($q_i = q^{\text{unsafe}}$) or covertness breaking ($q_i = \emptyset$) occurs. Finally, in Step 11, the edit function E is computed to satisfy opacity, covertness, and nonblockingness. It is important to note that the

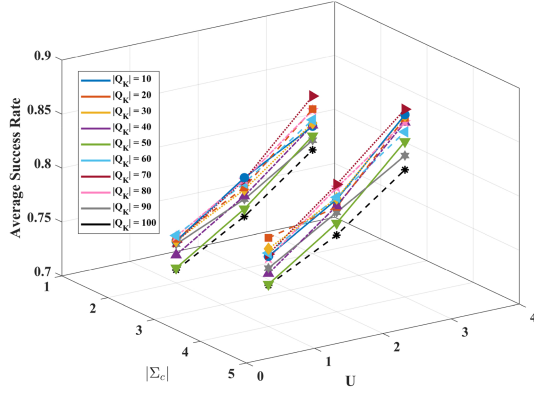


Fig. 11. Average success rate of finding nonempty solutions of 20 repeated simulations under different $|Q_K|$, $|\Sigma_c|$, and U for Procedure 1.

procedure will *converge and terminate within a finite number of steps*, despite the iterative nature of Steps 5–8. This is guaranteed by the following result.

Theorem 1: Procedure 1 terminates within finite steps.

Proof: The whole procedure's termination depends on the termination of iterative computation in Steps 5–8. We observe that whether the set Q_{del} is equal to $\emptyset$ is the key to determining the continuation of the iteration in Step 6. The worst case occurs when we only prune one state from $\mathcal{R}_S^j$ at each iteration. Furthermore, it is important to note that $\mathcal{R}_S^0$, which is the initial requirement considered in the iterative computation, is a finite-state automaton. Thus, $\mathcal{R}_S^j$ is also a finite-state automaton at each iteration. Hence, Steps 5–8 could terminate within a finite number of steps, and the worst-case number of iterations is $|Q_{\mathcal{R}_S^0}| \leq |Q_{\mathcal{R}}| \leq |Q_K| \times |Q_{CE}| \times |Q_{EC}| \times |Q_{SC}| \times |Q_I| \leq |Q_K| \times (2^{|\Sigma_c|} + 1) \times (U + 2) \times 2 \times (2^{|Q_K|} + 1)$. ■

Theorem 2: The computed nonempty S and E by using Procedure 1 ensure three properties: 1) $G||CE||EC||SC||I||E||S$ is nonblocking, 2) $P_{\Sigma}(L(G||CE||EC||SC||I||E||S)) \subseteq L(K)$, and 3) (E, S) is opaque and covert.

Proof: First, it is satisfied that $G||CE||EC||SC||I||E||S$ is nonblocking according to Step 11 of Procedure 1. Thus, property 1) holds. According to Steps 2 and 3, since the specification K has been encoded in the requirement $\mathcal{R}_S^0$, we have $L(G||CE||EC||SC||I||S) \subseteq L(G||K||CE||EC||SC||I)$. Hence, we have $L(G||CE||EC||SC||I||E||S) \subseteq L(G||K||CE||EC||SC||I)$, which implies that $P_{\Sigma}(L(G||CE||EC||SC||I||E||S)) \subseteq P_{\Sigma}(L(G||K||CE||EC||SC||I)) \subseteq L(K)$. Thus, property 2) holds. Notice that the requirement $\mathcal{R}_E^0$ is generated by removing the set of states Q_2 . Hence, it can be easily checked that the closed-loop system $G||CE||EC||SC||I||E||S$ will not reach any state in $\{(q, q_{ce}, q_{ec}, q_{sc}, q_i, q_e, q_s) \in Q \times Q_{CE} \times Q_{EC} \times Q_{SC} \times Q_I \times Q_E \times Q_S | q_I = q_{unsafe} \vee q_i = \emptyset\}$. According to Definitions 2 and 3, property 3) holds, which completes the proof. ■

Remark 4: Theorem 2 indicates that Procedure 1 is sound.

For Procedure 1, the time complexity depends on the synthesis steps (Steps 8 and 11) and the specific supervisor synthesis algorithm. For example, by using the synthesis algorithm in [5], the time complexity of Procedure 1 is $O(|\Sigma_P|2^{|Q_{\mathcal{R}_S^0}|} + \dots + |\Sigma_P| \times 2 + |\Sigma_P||Q_{\mathcal{R}_E^0}|2^{|Q_{\mathcal{R}_E^0}|}) = O(2|\Sigma_P|(2^{|Q_{\mathcal{R}_S^0}|} - 1) + |\Sigma_P||Q_{\mathcal{R}_E^0}|2^{|Q_{\mathcal{R}_E^0}|})$, where $|\Sigma_P| = |\Sigma| + |\Sigma_{s,E}| + 2^{|\Sigma_c|} + 2$, $|Q_{\mathcal{R}_S^0}| \leq |Q_K| \times (2^{|\Sigma_c|} + 1) \times (U + 2) \times 2 \times$

TABLE I
AVERAGE COMPUTATION TIME OF 20 REPEATED SIMULATIONS UNDER DIFFERENT $|Q_K|$ AND $|\Sigma_c|$ FOR PROCEDURE 1

$ Q_K $	$ \Sigma_c $	Average Computation Time (s)
10	2	0.002
	4	0.005
	6	0.009
	8	0.024
	10	0.072
20	2	0.011
	4	0.038
	6	0.079
	8	0.253
	10	0.611
30	2	0.392
	4	0.581
	6	0.897
	8	1.433
	10	2.146
40	2	1.018
	4	1.587
	6	2.362
	8	3.581
	10	5.197
50	2	2.768
	4	3.712
	6	5.269
	8	7.692
	10	11.124
60	2	5.423
	4	8.994
	6	13.683
	8	19.537
	10	27.261
70	2	10.717
	4	16.136
	6	23.025
	8	33.017
	10	45.023
80	2	17.483
	4	26.037
	6	37.017
	8	50.985
	10	65.441
90	2	30.592
	4	42.118
	6	55.927
	8	73.014
	10	91.436
100	2	47.467
	4	61.217
	6	78.848
	8	98.913
	10	121.532

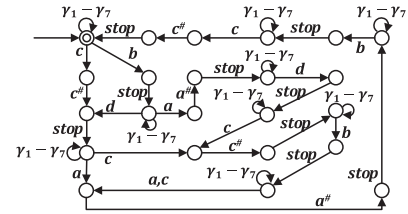


Fig. 12. Synthesized E (first E then S).

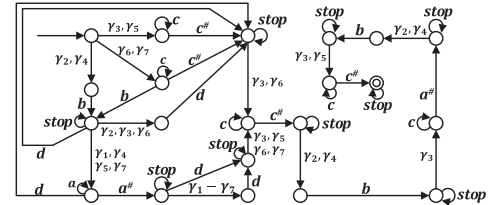


Fig. 13. Synthesized S (first E then S).

$(2^{|Q_K|} + 1)$, and $|Q_{\mathcal{R}_E^0}| \leq |Q| \times (2^{|\Sigma_c|} + 1) \times (U + 2) \times 2 \times (2^{|Q_K|} + 1) \times |Q_S|$.

Example 6: Let us continue the location privacy example. By adopting Procedure 1, the synthesized S and E are illustrated in Figs. 8 and 9. We first present a summary for the action sequence executed by the vehicle and the corresponding edited observation sequence at the intruder side. There are following cases: 1) the vehicle executes $ccbc$ and the intruder observes $cabca$ (or $cabc$), 2) the vehicle executes $ccbc$ and the intruder

TABLE II
AVERAGE SUCCESS RATE OF FINDING NONEMPTY SOLUTIONS OF 20
REPEATED SIMULATIONS UNDER DIFFERENT $|Q_K|$, $|\Sigma_c|$, AND U FOR
PROCEDURE 1

$ Q_K $	$ \Sigma_c $	U	Average Success Rate
10	2	1	0.739
		2	0.784
		3	0.818
	4	1	0.765
		2	0.804
		3	0.869
20	2	1	0.741
		2	0.775
		3	0.834
	4	1	0.782
		2	0.797
		3	0.866
30	2	1	0.739
		2	0.773
		3	0.821
	4	1	0.772
		2	0.805
		3	0.868
40	2	1	0.726
		2	0.767
		3	0.819
	4	1	0.749
		2	0.798
		3	0.862
50	2	1	0.714
		2	0.755
		3	0.809
	4	1	0.739
		2	0.782
		3	0.844
60	2	1	0.744
		2	0.779
		3	0.824
	4	1	0.768
		2	0.806
		3	0.853
70	2	1	0.738
		2	0.782
		3	0.846
	4	1	0.767
		2	0.818
		3	0.874
80	2	1	0.743
		2	0.782
		3	0.833
	4	1	0.763
		2	0.815
		3	0.862
90	2	1	0.736
		2	0.764
		3	0.805
	4	1	0.754
		2	0.792
		3	0.831
100	2	1	0.712
		2	0.748
		3	0.796
	4	1	0.738
		2	0.771
		3	0.818

observes *ccbabc* (or *ccbab*), 3) the vehicle executes *bdcbebc* and the intruder observes *bdcbebc* (or *bdcbab*), 4) the vehicle executes *bdcbebc* and the intruder observes *bdabcba* (or *bdabcb*), 5) the vehicle executes *badcbebc* and the intruder observes *bdcbebc* (or *bdcbab*), 6) the vehicle executes *badcbebc* and the intruder observes *bdabcba* (or *bdabcb*), and 7) the vehicle executes *badcbebc* and the intruder observes *badcbebc* (or *badcbab*). Due to the page limit, we shall only take case 1 as an instance to explain the cooperation between the supervisor (remote controller) and the edit function to achieve the privacy-preserving control. Initially, the control command $\gamma_3 = \{c\}$ or $\gamma_5 = \{a, c\}$ or $\gamma_6 = \{b, c\}$ or $\gamma_7 = \{a, b, c\}$ is issued by the remote controller, then the vehicle executes *c* (go straight), which is kept intact by the edit function *E* and would be observed by the remote controller. Then, the remote controller issues the control command $\gamma_3 = \{c\}$ or $\gamma_6 = \{b, c\}$, resulting in that the vehicle executes the event *c* (go straight). Then, *c* is changed into *a* by the edit function *E* and would be observed by the remote controller. Upon observing *a*, the control command $\gamma_2 = \{b\}$ or $\gamma_4 = \{a, b\}$ or $\gamma_6 = \{b, c\}$ or $\gamma_7 = \{a, b, c\}$ is issued, resulting in that the vehicle executes *b* (turn left). After observing *b*, the remote controller sends the control command $\gamma_3 = \{c\}$. Then, the vehicle executes *c* (go straight), which is kept intact by the edit function *E* and

Procedure 2.

Input: $G, CE, EC, SC, \mathcal{I}, K, (\Sigma_{E, \text{con}}, \Sigma_{E, \text{obs}}), (\Sigma_{S, \text{con}}, \Sigma_{S, \text{obs}})$.

Output: E, S .

1. Compute $\mathcal{P} = G||CE||EC||SC||\mathcal{I} = (Q_{\mathcal{P}}, \Sigma_{\mathcal{P}}, \xi_{\mathcal{P}}, q_{\mathcal{P}}^{\text{init}}, Q_{\mathcal{P}, m})$.
2. Generate $\mathcal{R}_E := (\mathcal{P})|_{Q_{\mathcal{P}} - Q_3} = (Q_{\mathcal{R}_E}, \Sigma_{\mathcal{R}_E}, \xi_{\mathcal{R}_E}, q_{\mathcal{R}_E}^{\text{init}}, Q_{\mathcal{R}_E, m})$, where
 - $Q_3 := \{(q, q_{ce}, q_{ec}, q_{sc}, q_i) \in Q_{\mathcal{P}} \mid q_i = q^{\text{unsafe}} \vee q_i = \emptyset\}$.
3. Synthesize a supervisor $E = (Q_E, \Sigma_E, \xi_E, q_E^{\text{init}}, Q_{E, m})$ over the edit function-control constraint $(\Sigma_{E, \text{con}}, \Sigma_{E, \text{obs}})$ by treating $\mathcal{P}$ as the plant and $\mathcal{R}_E$ as the requirement such that $\mathcal{P}||E$ is marker-reachable and $L(\mathcal{P}||E) \subseteq L(\mathcal{R}_E)$. If *E* exists, go to Step 4; otherwise, end the procedure.
4. Compute $\mathcal{P}_S = G||CE||EC||SC||\mathcal{I}||E$.
5. Compute $\mathcal{R}_S = G||K||CE||EC||SC||\mathcal{I}||E = (Q_{\mathcal{R}_S}, \Sigma_{\mathcal{R}_S}, \xi_{\mathcal{R}_S}, q_{\mathcal{R}_S}^{\text{init}}, Q_{\mathcal{R}_S, m})$.
6. Synthesize a supervisor $S = (Q_S, \Sigma_S, \xi_S, q_S^{\text{init}}, Q_{S, m})$ over the supervisor-control constraint $(\Sigma_{S, \text{con}}, \Sigma_{S, \text{obs}})$ by treating $\mathcal{P}_S$ as the plant and $\mathcal{R}_S$ as the requirement such that $\mathcal{P}_S||S$ is nonblocking and $L(\mathcal{P}_S||S) \subseteq L(\mathcal{R}_S)$.

would be observed by the remote controller. Then, the remote controller issues the control command $\gamma_2 = \{b\}$ or $\gamma_4 = \{a, b\}$, enabling the vehicle to execute *b* (turn left). Afterward, the remote controller observes *b* and sends the control command $\gamma_3 = \{c\}$ or $\gamma_5 = \{a, c\}$, under which the vehicle executes *c* (go straight). In order to remain covert, the edit function *E* has to either delete *c* or change *c* into *a*.

We also perform 20 repeated simulations for Procedure 1 under different $|Q_K|$ and $|\Sigma_c|$ as the time complexity is doubly exponential w.r.t. $|Q_K|$ and $|\Sigma_c|$. For simplicity, we set $U = 1$. Based on an Intel Core i5-9500 CPU, Table I presents the average computation time, which is illustrated in Fig. 10. It can be seen that, although the time complexity of Procedure 1 is doubly exponential w.r.t. $|Q_K|$ and $|\Sigma_c|$, Procedure 1 is practically scalable to systems with nontrivial state sizes. The reason is that, since not all events are defined for each state in the components CE, EC, SC, some states in the Cartesian product of the state set of these components are pruned in their parallel composition. Thus, the state set size we need to deal with in the actual computation is (much) smaller than the product of the state set sizes of the components *K*, CE, EC, SC, and $\mathcal{I}$ presented in the time complexity. We also perform 20 repeated simulations for Procedure 1 under different $|Q_K|$, $|\Sigma_c|$, and U . Table II presents the average success rate of finding nonempty solutions, which is illustrated in Fig. 11. It can be seen that the success rate of finding nonempty solutions for Procedure 1 is about 70%–80%. Besides, the higher $|\Sigma_c|$ (U , respectively), the higher the success rate of finding nonempty solutions as the number of control commands (the number of editing operations, respectively) increases, providing a larger feasible solution space.

B. Incremental Synthesis: First *E* Then *S*

Initially, the synthesis procedure focuses on constructing the edit function to guarantee the closed-loop system's opacity, covertness, and marker reachability. Subsequently, the supervisor is synthesized, achieving the enforcement of safety according

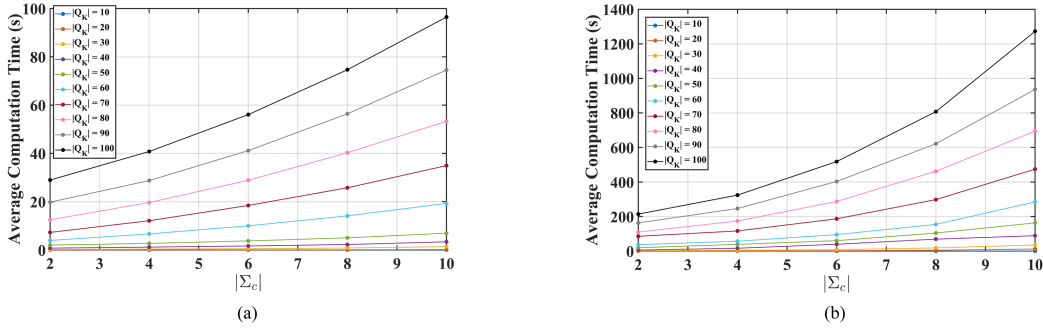


Fig. 14. Illustrative comparison of average computation time of 20 repeated simulations under different $|Q_K|$ and $|\Sigma_c|$. (a) Procedure 2. (b) Approach of Tai et al. [28].

TABLE III

AVERAGE COMPUTATION TIME OF 20 REPEATED SIMULATIONS UNDER DIFFERENT $|Q_K|$ AND $|\Sigma_c|$ FOR PROCEDURE 2 AND THE APPROACH OF TAI ET AL. [28]

$ Q_K $	$ \Sigma_c $	Average Computation Time (s)	
		Procedure 2	The approach of Tai et al. [28]
10	2	0.001	0.007
	4	0.004	0.018
	6	0.007	0.054
	8	0.017	0.147
	10	0.043	0.388
20	2	0.007	0.421
	4	0.017	0.981
	6	0.043	2.283
	8	0.082	5.461
	10	0.164	12.498
30	2	0.221	2.749
	4	0.389	4.775
	6	0.545	8.614
	8	0.833	18.795
	10	1.387	35.623
40	2	0.778	6.139
	4	1.187	16.774
	6	1.673	39.745
	8	2.256	69.473
	10	3.379	89.438
50	2	2.017	19.671
	4	2.736	38.994
	6	3.759	60.586
	8	5.048	104.879
	10	6.949	163.617
60	2	3.957	37.295
	4	6.658	57.446
	6	10.014	95.695
	8	14.139	154.894
	10	19.284	284.887
70	2	7.285	85.894
	4	12.118	116.812
	6	18.447	187.168
	8	25.769	298.395
	10	34.948	474.576
80	2	12.518	110.396
	4	19.649	174.559
	6	28.883	287.481
	8	40.274	462.483
	10	53.291	694.582
90	2	19.794	163.274
	4	28.752	246.591
	6	41.167	403.295
	8	56.383	621.487
	10	74.542	935.915
100	2	28.963	214.395
	4	40.783	324.287
	6	56.075	518.376
	8	74.649	807.194
	10	96.476	1272.483

TABLE IV

AVERAGE SUCCESS RATE OF FINDING NONEMPTY SOLUTIONS OF 20 REPEATED SIMULATIONS UNDER DIFFERENT $|Q_K|$, $|\Sigma_c|$, AND U FOR PROCEDURE 2 AND THE APPROACH OF TAI ET AL. [28]

$ Q_K $	$ \Sigma_c $	U	Average Success Rate	
			Procedure 2	The approach of Tai et al. [28]
10	2	1	0.713	0.531
		2	0.754	0.574
		3	0.788	0.603
	4	1	0.729	0.547
		2	0.783	0.595
		3	0.812	0.618
20	2	1	0.718	0.537
		2	0.745	0.569
		3	0.793	0.597
	4	1	0.746	0.558
		2	0.781	0.593
		3	0.826	0.614
30	2	1	0.732	0.525
		2	0.773	0.553
		3	0.804	0.594
	4	1	0.759	0.551
		2	0.791	0.576
		3	0.834	0.618
40	2	1	0.728	0.542
		2	0.761	0.578
		3	0.808	0.614
	4	1	0.745	0.568
		2	0.793	0.602
		3	0.844	0.637
50	2	1	0.722	0.539
		2	0.756	0.564
		3	0.794	0.594
	4	1	0.753	0.561
		2	0.786	0.587
		3	0.824	0.628
60	2	1	0.731	0.542
		2	0.764	0.577
		3	0.798	0.608
	4	1	0.766	0.565
		2	0.782	0.593
		3	0.813	0.634
70	2	1	0.715	0.528
		2	0.738	0.562
		3	0.785	0.599
	4	1	0.736	0.551
		2	0.772	0.596
		3	0.828	0.627
80	2	1	0.726	0.548
		2	0.768	0.581
		3	0.807	0.622
	4	1	0.753	0.574
		2	0.782	0.607
		3	0.833	0.649
90	2	1	0.736	0.528
		2	0.779	0.565
		3	0.814	0.597
	4	1	0.758	0.548
		2	0.796	0.589
		3	0.834	0.614
100	2	1	0.748	0.533
		2	0.765	0.572
		3	0.793	0.605
	4	1	0.767	0.562
		2	0.795	0.593
		3	0.831	0.623

to the specification as well as ensuring nonblockingness. The specific details of the proposed algorithm are outlined as follows.

Steps 1–3 of the procedure deal with computing the edit function, and the supervisor is computed in Steps 4–6. In the edit function synthesis, we need to construct the surrogate plant $\mathcal{P}$ and the surrogate requirement $\mathcal{R}_E$. By eliminating from $\mathcal{P}$ the state set Q_3 where the intruder infers the secret ($q_i = q^{\text{unsafe}}$) or the edit function is exposed ($q_i = \emptyset$), $\mathcal{R}_E$ can be obtained. In Step 3, the objective is to compute the edit function E , which ensures both $L(\mathcal{P}||E) \subseteq L(\mathcal{R}_E)$ and marker reachability. As for the synthesis of the supervisor, we construct the surrogate plant

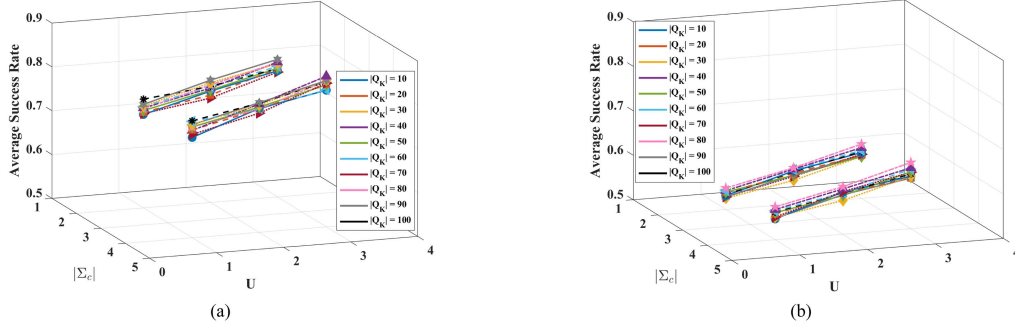


Fig. 15. Illustrative comparison of average success rate of finding nonempty solutions of 20 repeated simulations under different $|Q_K|$, $|\Sigma_c|$, and U . (a) Procedure 2. (b) Approach of Tai et al. [28].

$\mathcal{P}_S$ based on the previously synthesized edit function E from Steps 1 to 3. The surrogate requirement $\mathcal{R}_S$ is determined in Step 5 by combining $\mathcal{P}_S$ with the specification K . We incorporate K in the supervisor synthesis part because it can be satisfied only under suitable control commands issued by the supervisor. Finally, in Step 6, the supervisor S is computed to ensure both $L(\mathcal{P}_S||S) \subseteq L(\mathcal{R}_S)$ and nonblockingness. Procedure 2 is guaranteed to terminate within finite steps as iterative computations are not involved.

Theorem 3: The computed nonempty E by using Procedure 2 combined with any supervisor $\tilde{S}$ is an opaque and covert edit function–supervisor pair.

Proof: Notice that the requirement $Q_{\mathcal{R}_E}$ is generated by removing the state set Q_3 . Hence, it can be checked that $\mathcal{P}||E$ will not reach any state in $\{(q, q_{ce}, q_{ec}, q_{sc}, q_i, q_e) \in Q \times Q_{CE} \times Q_{EC} \times Q_{SC} \times Q_I \times Q_E | q_i = q_{\text{unsafe}} \vee q_i = \emptyset\}$. As a result, $\mathcal{P}||E||\tilde{S}$ will not reach any state where the secret state is inferred or the edit function is exposed. Based on Definitions 2 and 3, the proof is completed. ■

Theorem 4: The computed nonempty E and S by using Procedure 2 ensure three properties: 1) $G||CE||EC||SC||I||E||S$ is nonblocking, 2) $P_\Sigma(L(G||CE||EC||SC||I||E||S)) \subseteq L(K)$, and 3) (E, S) is opaque and covert.

Proof: First, it is satisfied that $G||CE||EC||SC||I||E||S$ is nonblocking according to Step 6 of Procedure 2. Thus, the property 1) holds. According to Step 6, we have $L(G||CE||EC||SC||I||E||S) \subseteq L(G||K||CE||EC||SC||I||E)$. Hence, we have $P_\Sigma(L(G||CE||EC||SC||I||E||S)) \subseteq P_\Sigma(L(G||K||CE||EC||SC||I||E)) \subseteq L(K)$. Thus, property 2) holds. In addition, theorem 3 indicates that the computed edit function–supervisor pair is opaque and covert. Thus, the property 3) holds, which completes the proof. ■

Remark 5: Theorem 4 indicates that Procedure 2 is sound.

For Procedure 2, the time complexity depends on the synthesis steps (Steps 3 and 6) and the specific supervisor synthesis algorithm. For example, by using the synthesis algorithm in [5], the time complexity of Procedure 2 is $O(|\Sigma_P|2^{|Q_{\mathcal{R}_E}|} + |\Sigma_P||Q_{\mathcal{R}_S}|2^{|Q_{\mathcal{R}_S}|})$, where $|\Sigma_P| = |\Sigma| + |\Sigma_{S,E}| + 2^{|\Sigma_c|} + 2$, $|Q_{\mathcal{R}_E}| \leq |Q| \times (2^{|\Sigma_c|} + 1) \times (U + 2) \times 2 \times (2^{|Q_K|} + 1)$, and $|Q_{\mathcal{R}_S}| \leq |Q_K| \times (2^{|\Sigma_c|} + 1) \times (U + 2) \times 2 \times (2^{|Q_K|} + 1) \times |Q_E|$.

Example 7: Let us continue the location privacy example. By adopting Procedure 2, the synthesized E and S are shown in Figs. 12 and 13. For clarity, we still first present a summary for the action sequence executed by the vehicle and the corresponding edited observation sequence at the intruder side based on the privacy-preserving control strategies embedded in the

synthesized E and S . There are following cases: 1) the vehicle executes *ccbcbe* and the intruder observes *ccbabcb*, 2) the vehicle executes *badebcbe* and the intruder observes *badebabcb*, and 3) the vehicle executes *bdcbebc* and the intruder observes *bdcbabcb*. It can be checked that all of these strategies could ensure 1) the safety enforced by the specification K and nonblockingness are satisfied, 2) the covertness is enforced, and 3) the intruder cannot infer that the vehicle has reached the secret location 12, i.e., opacity enforcement. More detailed explanations will not be presented due to space limit but the computed strategy could be interpreted similarly to the one explained in Example 6.

We also perform 20 repeated simulations for Procedure 2 under different $|Q_K|$ and $|\Sigma_c|$. Table III presents the average computation time, which is illustrated in Fig. 14(a). It can be seen that Procedure 2 is scalable to systems with nontrivial state sizes. The reason is similar to that of Procedure 1. We also perform 20 repeated simulations for Procedure 2 under different $|Q_K|$, $|\Sigma_c|$, and U , and the average success rate of finding nonempty solutions is presented in Table IV, which is illustrated in Fig. 15(a). It can be seen that the higher $|\Sigma_c|$ (U , respectively), the higher the success rate of finding nonempty solutions due to the similar reason presented in Section V-A for Procedure 1.

To show that Procedure 2 outperforms the approach of Tai et al. [28], which is the one most related to our work, we conduct comparative simulations w.r.t. the computation time and the success rate of finding nonempty solutions. For the sake of fairness, we only compare [28] and Procedure 2 because they adopt the same synthesis order, while Procedure 1 adopts the reverse synthesis order. Besides, since the work of Tai et al. [28] did not consider the specification, it is removed when we conduct simulations using the approach of Tai et al. [28]. The last column of Table III (Table IV, respectively) presents the computation time (the success rate of finding nonempty solutions, respectively) for the approach of Tai et al. [28], which is illustrated in Fig. 14(b) [see Fig. 15(b), respectively]. By comparison, Procedure 2 is more time efficient and has better scalability than the approach of Tai et al. [28]. The success rate of finding nonempty solutions of Procedure 2 is about 70%–80%, while the success rate of finding nonempty solutions of the approach of Tai et al. [28] is only about 50%–60%. Thus, Procedure 2 has a higher success rate of finding nonempty solutions than the approach of Tai et al. [28].

VI. CONCLUSION

This article centers on a supervisory control problem with a privacy-preserving objective. The aim is to guarantee opacity

enforcement and specification satisfaction at the supervisory control layer. Our approach involves modeling the cosynthesis problem using the Ramadge–Wonham supervisory control framework in a distributed manner, and designing two innovative heuristic techniques to compute an edit function and supervisor pair. This formulation enables the utilization of various existing synthesis tools. In the future, we plan to improve the solutions by exploring more advanced distributed synthesis approaches. In addition, the decidability of the privacy-preserving supervisory control problem is also worthy of investigation. Furthermore, the case where the intruder also knows the supervisor model is theoretically more challenging, which needs more substantial efforts on the cosynthesis algorithm development.

REFERENCES

- [1] F. Farivar, M. Haghighi, A. Jolfaei, and M. Alazab, “Artificial intelligence for detection estimation and compensation of malicious attacks in nonlinear cyber-physical systems and industrial IoT,” *IEEE Trans. Ind. Informat.*, vol. 16, no. 4, pp. 2716–2725, Apr. 2020.
- [2] Q. Sun, K. Zhang, and Y. Shi, “Resilient model predictive control of cyber-physical systems under DoS attacks,” *IEEE Trans. Ind. Informat.*, vol. 16, no. 7, pp. 4920–4927, Jul. 2020.
- [3] M. M. Hassan, A. Gumaei, S. Huda, and A. Almogren, “Increasing the trustworthiness in the industrial IoT networks through a reliable cyberattack detection model,” *IEEE Trans. Ind. Informat.*, vol. 16, no. 9, pp. 6154–6162, Sep. 2020.
- [4] L. Mazare, “Using unification for opacity properties,” in *Proc. Workshop Issues Theory Secur.*, 2004, pp. 165–176.
- [5] W. M. Wonham and K. Cai, *Supervisory Control of Discrete-Event Systems*. Cham, Switzerland: Springer, 2019.
- [6] C. Cassandras and S. Lafortune, *Introduction to Discrete Event Systems*, 3rd ed. New York, NY, USA: Springer, 2021.
- [7] E. Badouel, M. Bednarczyk, A. Borzyszkowski, B. Caillaud, and P. Darondeau, “Concurrent secrets,” *Discrete Event Dyn. Syst.*, vol. 17, no. 4, pp. 425–446, 2007.
- [8] J. Dubreil, P. Darondeau, and H. Marchand, “Supervisory control for opacity,” *IEEE Trans. Autom. Control*, vol. 55, no. 5, pp. 1089–1100, May 2010.
- [9] S. Takai and R. Kumar, “Verification and synthesis for secrecy in discrete-event systems,” in *Proc. Amer. Control Conf.*, 2009, pp. 4741–4746.
- [10] A. Saboori and C. N. Hadjicostis, “Opacity-enforcing supervisory strategies via state estimator constructions,” *IEEE Trans. Autom. Control*, vol. 57, no. 5, pp. 1155–1165, May 2012.
- [11] X. Yin and S. Lafortune, “A uniform approach for synthesizing property-enforcing supervisors for partially-observed discrete-event systems,” *IEEE Trans. Autom. Control*, vol. 61, no. 8, pp. 2140–2154, Aug. 2016.
- [12] Y. Tong, Z. Li, C. Seatzu, and A. Giua, “Current-state opacity enforcement in discrete event systems under incomparable observations,” *Discrete Event Dyn. Syst.*, vol. 28, pp. 161–182, 2018.
- [13] J. Hou, X. Yin, S. Li, and M. Zamani, “Abstraction-based synthesis of opacity-enforcing controllers using alternating simulation relations,” in *Proc. 58th IEEE Conf. Decis. Control*, 2019, pp. 7653–7658.
- [14] Y.-C. Wu and S. Lafortune, “Synthesis of insertion functions for enforcement of opacity security properties,” *Automatica*, vol. 50, no. 5, pp. 1336–1348, 2014.
- [15] R. J. Barcelos and J. C. Basilio, “Enforcing current-state opacity through shuffle in event observations,” in *Proc. 14th Int. Workshop Discrete Event Syst.*, 2018, pp. 106–111.
- [16] Y. Ji, Y.-C. Wu, and S. Lafortune, “Enforcement of opacity by public and private insertion functions,” *Automatica*, vol. 93, pp. 369–378, 2018.
- [17] S. Mohajerani, Y. Ji, and S. Lafortune, “Compositional and abstraction-based approach for synthesis of edit functions for opacity enforcement,” *IEEE Trans. Autom. Control*, vol. 65, no. 8, pp. 3349–3364, Aug. 2019.
- [18] Y. Falcone and H. Marchand, “Enforcement and validation (at runtime) of various notions of opacity,” *Discrete Event Dyn. Syst.*, vol. 25, no. 4, pp. 531–570, 2015.
- [19] F. Cassez, J. Dubreil, and H. Marchand, “Synthesis of opaque systems with static and dynamic masks,” *Formal Methods Syst. Des.*, vol. 40, no. 1, pp. 88–115, 2012.
- [20] X. Yin and S. Li, “Synthesis of dynamic masks for infinite-step opacity,” *IEEE Trans. Autom. Control*, vol. 65, no. 4, pp. 1429–1441, Apr. 2020.
- [21] B. Zhang, S. Shu, and F. Lin, “Maximum information release while ensuring opacity in discrete event systems,” *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 3, pp. 1067–1079, Jul. 2015.
- [22] S. Lafortune, F. Lin, and C. N. Hadjicostis, “On the history of diagnosability and opacity in discrete event systems,” *Annu. Rev. Control*, vol. 45, pp. 257–266, 2018.
- [23] J. Wang, S. Baldi, W. Yu, and X. Yin, “Distributed fault diagnosis in discrete event systems with transmission delay impairments,” *IEEE Trans. Autom. Control*, vol. 69, no. 8, pp. 5508–5515, Aug. 2024.
- [24] J. Kim, H. Shim, and K. Han, “Dynamic controller that operates over homomorphically encrypted data for infinite time horizon,” *IEEE Trans. Autom. Control*, vol. 68, no. 2, pp. 660–672, Feb. 2023.
- [25] R. Fritz, M. Fauser, and P. Zhang, “Controller encryption for discrete event systems,” in *Proc. Amer. Control Conf.*, 2019, pp. 5633–5638.
- [26] S. Zhou, Z. Yu, E. S. A. Nasr, H. A. Mahmoud, E. M. Awwad, and N. Wu, “Homomorphic encryption of supervisory control systems using automata,” *IEEE Access*, vol. 8, pp. 147185–147198, 2020.
- [27] P. M. Lima, L. K. Carvalho, and M. V. Moreira, “Confidentiality of cyber-physical systems using event-based cryptography,” *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 1735–1740, 2020.
- [28] R. Tai, L. Lin, Y. Zhu, and R. Su, “Privacy-preserving co-synthesis against sensor-actuator eavesdropping intruder,” *Automatica*, vol. 150, 2023, Art. no. 110860.
- [29] Y.-C. Wu, K. A. Sankararaman, and S. Lafortune, “Ensuring privacy in location-based services: An approach based on opacity enforcement,” in *Proc. 14th Int. Workshop Discrete Event Syst.*, 2014, pp. 33–38.
- [30] J. Thistle, “Undecidability in decentralized supervision,” *Syst. Control Lett.*, vol. 54, no. 5, pp. 503–509, 2005.